Application of Machine Learning on Futures Trading

UIUC MSFE

Zilin Mao Zhaolin Shi

Instructor: Matthew Murphy

December 3, 2025

Abstract

This report develops an end-to-end machine-learning pipeline for systematic futures trading in Chinese commodity markets. We combine rollover-adjusted continuous prices, robust feature engineering and preprocessing, leakage-aware walk-forward training, model-driven feature selection, and score-to-signal execution with capital-based backtesting. Using daily data from 2014–2023 across a diversified set of liquid contracts, we compare Random Forest, XGBoost, and an MLP in both cross-sectional multi-contract and single-contract settings. We find that model choice and feature-selection stability materially impact signal quality and out-of-sample P&L: MLP/XGBoost are more effective in several single-contract tests, while multi-contract portfolios are more robust overall, with RF/MLP delivering the strongest risk-adjusted performance. These results highlight the importance of regime sensitivity and factor-structure dependence when deploying nonlinear models in futures trading.

Contents

1 Introduction **3**
1.1 Project Motivation . . . 3
1.2 Key Contributions . . . 3

2 Data **3**
2.1 Data attributions . . . 3
2.2 Backward Ratio Adjustment . . . 3
2.2.1 Reasons . . . 3
2.2.2 Implementation . . . 4

3 Methodology **4**
3.1 Single Contract Pipeline . . . 4
3.2 Multiple Contracts Pipeline . . . 6
3.3 Multiple contract models implementation . . . 8

4 Results **10**
4.1 Single-Contract Backtest Performance (RF) . . . 10
4.2 Single-Contract Backtest Performance (XGB) . . . 11
4.3 Single-Contract Backtest Performance (MLP) . . . 13
4.4 Multiple-Contract Backtest Performance . . . 14

5 Summary **16**
5.1 Models Comparison on Single Contract . . . 16
5.2 Models Comparison on Multiple Contract . . . 17
5.3 Future Works . . . 17

1 Introduction

1.1 Project Motivation

- **Market setting:** Chinese commodity futures are highly liquid and sector-diverse, yet exhibit frequent regime shifts (policy, supply shocks, structural breaks), making signal extraction challenging but economically meaningful.
- **Why machine learning:** Linear factor models often miss nonlinear interactions among OHLC geometry, momentum, volatility dynamics, and volume/open-interest positioning. We therefore study three nonlinear learners—**Random Forest**, **XGBoost**, and **MLP**—and evaluate how inductive bias translates into feature selection, signal strength, and trading performance.
- **Practical constraints:** Reliable evaluation requires (i) roll-adjusted continuous prices to remove mechanical jumps, (ii) robust preprocessing for heavy tails/outliers, (iii) imbalance-aware learning for extreme-move labels, and (iv) strict leakage control in time-series training and preprocessing.

1.2 Key Contributions

- **End-to-end research engine:** Implemented a complete futures ML workflow covering data acquisition, rollover adjustment, factor engineering, leakage-safe preprocessing, walk-forward training, factor selection, direction inference, score construction, and capital-based backtesting with risk diagnostics.
- **Model and setting comparison:** Conducted controlled experiments across **multi-contract** (cross-sectional long/short) and **single-contract** pipelines, benchmarking RF/XGB/MLP under consistent labeling, execution, and evaluation rules to isolate how model-driven factor selection impacts out-of-sample returns and tail risk.

2 Data

2.1 Data attributions

We construct our dataset using daily main-continuous futures prices from the Akshare API for Chinese commodity markets. The sample covers 12 highly liquid contracts—RB0, HC0, CU0, MA0, TA0, FG0, RM0, AU0, AG0, CF0, SR0, and RU0—from 1 January 2014 to 31 December 2023. For each contract, we retrieve daily open, high, low, close, trading volume, open interest, and (where available) settlement prices as the raw inputs for return, volatility, volume, and term–structure factor construction. These contracts are chosen to span major sectors (ferrous metals, non–ferrous metals, chemicals, precious metals, agriculture, and rubber) while maintaining sufficient liquidity and a consistent 10-year history for factor construction and backtesting

2.2 Backward Ratio Adjustment

2.2.1 Reasons

For each futures symbol we work with a main-continuous price series that switches delivery month when market liquidity migrates. These contract rolls typically introduce discrete price jumps that are driven by contract specifications and term-structure effects (e.g., different maturities, carry, and storage costs), rather than by true changes in the underlying economic value.

If we use raw main-continuous prices, these artificial jumps will:

- contaminate return calculations around roll dates,
- distort momentum, volatility, and technical indicators,
- create spurious "signals" that are purely mechanical,

To avoid this, we apply a backward ratio adjustment at each roll date so that the continuous price series is smooth across rolls and all return-based features reflect genuine market dynamics instead of contract mechanics.

2.2.2 Implementation

We use AkShare's `get_roll_yield_bar` function for each underlying (e.g. `var = "RB"` for RB0) to obtain daily term-structure information. From this data we:

1. Extract the daily main contract (`near_by`) and identify days when it changes relative to the previous trading day.
2. Mark these days as roll dates and record the contract pair `from_contract` (old main) and `to_contract` (new main).

Given the raw main-continuous daily series for each symbol (open, high, low, close, and, if available, settle), we then perform a backward ratio adjustment:

1. Sort the data by date and initialize a cumulative adjustment factor $F = 1$.
2. Traverse the series backward in time (from the most recent date to the earliest).
3. On each roll date t_i, compute a price ratio using close prices around the roll, e.g.

$$r_i = \frac{C_{\text{new}}(t_i)}{C_{\text{old}}(t_i^-)}.$$

4. Update the cumulative factor $F \leftarrow F \times r_i$ and multiply all earlier prices by the updated factor F.

This procedure produces adjusted price fields (`adj_open`, `adj_high`, `adj_low`, `adj_close`, and `adj_settle` when available) that are continuous at each roll date. All downstream return calculations and factors are then computed on these ratio-adjusted series.

3 Methodology

3.1 Single Contract Pipeline

1. Feature Engineering

We engineer features from two complementary sources: price action (OHLC geometry + trends) and flow/positioning (volume + open interest). From OHLC bars we compute r_t, body-to-range ratio, upper/lower shadow ratios, and bull/bear flags, then extend them to multi-horizon momentum signals with slope and acceleration terms, as well as realized volatility dynamics via rolling $\sigma(r)$, volatility multi-slope, and volatility acceleration. Trend and mean-reversion forces are captured using MA_n, price-to-MA bias, MA multi-slope, MA acceleration, and angle-like descriptors.

On the flow side, we use volume and OI returns, their moving averages and biases, vol-of-vol, and volume–OI ratios as proxies for participation and positioning. We further introduce

interaction features, like price–volume, price–OI divergence, rolling correlations, absorption and effort–result measures to separate "easy trend" moves from crowded or fading regimes.

2. Missing Value Handling

- Forward-fill / median fill (train-only statistics)
- Log-transform heavy tails

3. Preprocessing

We apply a leakage-safe preprocessing pipeline fit on the training window only and then reused unchanged on the test window. Concretely, we first replace $\pm\infty$ with NaN and perform forward-fill to ensure all information comes from the past, then impute any remaining missing values using training medians. Next, we stabilize heavy-tailed factors by (i) automatically detecting high-kurtosis columns and (ii) manually including key flow variables (e.g., volume/hold); for these factors we apply a signed $\log(1 + |x|)$ transform after percentile-based clipping to reduce extreme spikes. For slope-type features, we further clip them using training mean and standard deviation bounds, and we add a final global clipping step per factor using training statistics (approximately $\mu \pm k\sigma$) to suppress outliers. Finally, we standardize all factors using either `StandardScaler` or `RobustScaler`, trained on the training set only, and transform both train/test with the same fitted scaler and stored clipping/imputation parameters.

4. Train–Test Split

- Time-series split: early 40% train, later 60% test

5. Multi-Model Feature Selection

We perform model-driven factor selection by training three nonlinear classifiers—Random Forest (RF), XGBoost (XGB), and a Neural Network (MLP)—to predict large up and large down events (y_{up} and y_{down}). To keep the downstream pipeline unchanged, all three implementations return the same outputs——permutation-importance ranking, selected factor list, and predicted probabilities), while handling class imbalance consistently (RF: `balanced_subsample`, XGB: `scale_pos_weight`, MLP: `pos_weight` in `BCEWithLogitsLoss`).

Factor selection follows a unified two-stage filter. (1) Permutation importance: after fitting a multi-factor model on the provided dataset, we measure the marginal contribution of each factor by shuffling that column n_{perm} times and averaging the performance drop, using `AUC` as the baseline metric whenever both classes exist (otherwise falling back to `--logloss`). We retain candidates above a data-adaptive threshold

$$\text{thr} \;=\; \max(\texttt{imp_min},\ \text{Quantile}(\texttt{importance},\ \texttt{imp_quantile}))\,.$$

and only keep the top `max_single_check` candidates for the next step. (2) Single-factor validation: we re-train a one-factor model for each candidate and require `AUC` $\geq$ `auc_single_min` to remove spurious "importance" caused by interactions or noise. The final factor set is capped by `top_k_max` (e.g., 25 factors per direction).

Because each learner has a different inductive bias, the resulting factor universe is model-dependent: RF tends to produce more stable rankings via bagging and nonlinear splits, XGB yields sharper and sparser rankings driven by boosting gains and interactions, while MLP can capture higher-order smooth interactions but is more sensitive to optimization and regularization. These selected factors are then passed to direction detection and score construction, so the model choice directly changes the score function and ultimately the backtested P&L profile.

6. Factor Direction Detection

We infer a leakage-safe direction sign (+1/–1/0) for each selected factor on the training set only using a robust ensemble: (i) qcut bucketing into n_{group} bins and a baseline sign from top–bottom

bucket mean forward returns, (ii) IC sign from Pearson correlation, and (iii) monotonicity from Spearman rank on bucket means; we also fit a centered linear regression and, when statistically significant (e.g., $|t| \geq 2$), use it as a constraint (contradiction $\Rightarrow$ veto). To ensure stability under time dependence, we run block bootstrap resampling (block size $\approx 5\%$ of samples) to estimate directional agreement and drop unstable factors via a confidence threshold (e.g., $< 0.7 \Rightarrow$ sign 0). Finally, we build `sign_dict` for long/short lists and remove zero-sign factors while preserving the original importance order, producing direction-aligned factor sets for score construction.

7. Score Function

Using the direction-aligned factor sets {`final_up_factors`, `final_down_factors`} and their sign dictionaries, we compute `score_long`, `score_short`, and the net `score` via `add_score` on both the training and test samples. Each factor is standardized with a rolling window into z-scores and winsorized to control heavy tails; we optionally use robust scaling for stability. To avoid redundancy, highly correlated factors are removed ($\rho > 0.7$), and a conflict-suppression rule neutralizes days where long/short evidence strongly disagrees (conflict threshold = 0.7). Finally, the long and short strength signals are aggregated into a smooth, continuous score that serves as the unified trading signal; we also log sanity checks by inspecting the effective factors actually used and the score distribution quantiles to support the later selection of the trading threshold T.

8. Backtest & Evaluation

We calibrate the trading threshold T on the training sample using the empirical distribution of $|score|$: $T_{\text{data}} = \text{Quantile}(|score|, q)$ and $T = \max(T_{\text{data}}, min_T)$, so weak signals are filtered while the trade frequency adapts to score dispersion. In parallel, we set the profit/loss scale `base_th` from training prices by estimating rolling daily volatility (lookback = 50), winsorizing daily returns, and scaling a reference threshold `base_th_ref` proportionally to the median volatility with floor and ceiling protection. We further construct weekly regime filters——trend via MA-fast/slow, volatility regime by weekly vol quantiles, and a crowding proxy based on OI and price changes——shifted by one week to avoid look-ahead; these filters only gate new entries.

We trade when $score > T$ (long) or $score < -T$ (short), execute at the next-day open via `shift_signal`, and size each position using a fixed margin fraction `pos_pct` with leverage (capital-based compounding). While in position, we update trade-level margin return cum_ret and exit using layered risk rules: hard take-profit ($cum_ret \geq 2 *$ `base_th`), stop-loss ($cum_ret \leq -0.8 *$ `base_th`), pullback take-profit after reaching a first target ($cum_ret \geq$ `base_th` then retrace to $0.7 *$ `base_th`), and a maximum holding period (`max_hold_days`). Performance is summarized by the equity curve and portfolio daily returns, annualized sharpe excessing over a risk-free rate 1.2%, and trade-level diagnostics including total trades, hit ratio, and worst-k losses.

3.2 Multiple Contracts Pipeline

1. Factor Universe and RF-Derived Top-25 Selection

We start from a library of 60 interpretable factors spanning (i) momentum (multi-horizon returns), (ii) moving-average deviation, (iii) volatility, (iv) volume/open-interest signals, and (v) technical indicators. To obtain a data-driven prior over factor relevance, we train a *simple Random Forest classifier* on the cross-sectional classification task and rank all candidate factors by feature importance. We then select the Top-25 factors with the highest importance scores (stored as `importance_order`), which serve as the candidate set used in downstream collinearity filtering and Top-15 construction.

3. Collinearity Removal (Greedy Correlation Pruning)

Since many technical indicators and multi-horizon returns are strongly correlated, we explicitly remove highly collinear factors to avoid redundancy and instability. Concretely, we compute the absolute Pearson correlation matrix on valid samples (after replacing $\pm\infty$ and dropping missing values), and apply a greedy pruning procedure in RF-importance order with threshold $\tau_\rho = 0.90$:

- Traverse `candidate_features` from most to least important.
- Keep factor f if it has not been dropped.
- Drop any remaining factor g such that $|\rho(f, g)| > \tau_\rho$.

This step preserves the highest-importance representative among groups of near-duplicate signals, producing a de-collinearized set `selected_all`. The exact implementation corresponds to the loop over `to_keep/to_drop` in Section 2.15.

4. Final Top-15 Set (Distinct Semantics) and Horizon Mapping

Finally, we take the first $N = 15$ factors from the original RF-importance order that survive the collinearity filter:

$$\texttt{selected_features_25} = \texttt{Top15}(\texttt{importance_order} \cap \texttt{selected_all}).$$

Intuitively, this procedure yields 15 factors that are both *highly predictive under a simple RF screening model* and *less redundant* (i.e., closer to covering different economic/technical meanings rather than repeating the same signal multiple times). For downstream consistency, we use the same factor set for all holding horizons $h \in \{1, 2, 3\}$:

$$\texttt{h_feature_map}[h] = \texttt{selected_features_25}, \quad h = 1, 2, 3.$$

5. Selected Top-15 Factors and Interpretation

In our implementation, the final Top-15 factor list (stored as `selected_features_25`) is:

- **R_5**: 5-day return (short-horizon momentum); captures short-term trend continuation or fast mean-reversion.
- **R_1**: 1-day return (ultra-short momentum); sensitive to micro-reversal and very recent shocks.
- **RHOLD_5**: 5-day open-interest change; proxies for positioning/flow and the build-up or unwind of exposures.
- **RANGE**: intraday price range (e.g., high–low scaled); measures realized dispersion and uncertainty.
- **Corr_PV_5**: 5-day price–volume correlation; indicates whether price moves are confirmed by volume (trend credibility).
- **RVOL_20**: 20-day relative volume; detects abnormal activity versus recent history (information arrival / crowding).
- **VOL_20**: 20-day realized volatility; medium-term risk-regime signal that affects return distributions.
- **RHOLD_20**: 20-day open-interest change; slower positioning trend linked to medium-term structure/carry shifts.
- **VOL_5**: 5-day realized volatility; short-term risk/uncertainty indicator responsive to recent jumps.
- **RVOL_5**: 5-day relative volume; short-term activity burst indicator often associated with transient momentum.
- **RSI_14**: 14-day RSI; technical overbought/oversold proxy for mean-reversion vs. trend behavior.
- **TURNOVER**: turnover / liquidity proxy; reflects trading intensity and potential transaction-cost/crowding conditions.

- **R_60**: 60-day return (intermediate momentum); captures medium-horizon trend persistence.
- **ATR_5**: 5-day Average True Range; robust volatility/range measure incorporating gap risk.
- **R_120**: 120-day return (longer momentum); captures longer-term trend and regime effects.

6. Global Standardization for MLP Inputs

For the MLP classifier, we standardize the selected Top-15 factors using a global `StandardScaler` (mean 0, standard deviation 1). Specifically, for each factor x we construct

$$x^{\text{std}} = \frac{x-\mu}{\sigma},$$

where (μ, σ) are estimated from the available dataset, and we write standardized columns back to `panel` with prefix `std_`. We then define

$$\texttt{h_feature_map_std}[h] = \{\texttt{std_}f : f \in \texttt{selected_features_25}\}, \quad h = 1, 2, 3.$$

7. Practical Note on Leakage Control

The implementation above performs global scaling for simplicity and numerical stability. For a strictly leakage-free evaluation, the scaler should be fit *within each rolling training window* and then applied unchanged to the corresponding test window, keeping the same factor set and model in

3.3 Multiple contract models implementation

All three classifiers (Random Forest, XGBoost, and MLP) are evaluated under the same cross-sectional rolling backtest protocol. The models differ only in the scoring function (*probability of up move* p_{up}), while the data construction, labeling, portfolio formation, and performance measurement are identical.

Data universe and volatility filter. Let the daily cross-section contain contracts indexed by $i \in \mathcal{U}_t$ on date t. We compute a rolling volatility proxy for each contract using a 20-day rolling standard deviation of the overnight return series $r^{\text{oo}}_{i,t}$:

$$\text{vol}_{i,t} = \text{Std}_{20}\big(r^{\text{oo}}_{i,\cdot}\big)\,.$$

Each day we retain only the top-N_{vol} contracts by $\text{vol}_{i,t}$ to form the tradable universe $\widetilde{\mathcal{U}}_t$.

Return definition and forward h-period target (overnight only). We use an overnight Open→Open return definition:

$$r^{\text{oo}}_{i,t} = \frac{\text{adj_open}_{i,t+1}}{\text{adj_open}_{i,t}} - 1.$$

For a given holding horizon h, we define the forward cumulative return target as:

$$R^{(h)}_{i,t} = \sum_{j=1}^{h} r^{\text{oo}}_{i,t+j},$$

which aligns with the implementation `rolling(h).sum().shift(-h)`, i.e. $R^{(h)}_{i,t}$ uses information from dates $t+1, \ldots, t+h$ only.

Cross-sectional labeling with neutrality band (`label_tau` = 0.002). Let the cross-sectional median of the forward return on day t be

$$m_t = \text{median}_{i \in \widetilde{\mathcal{U}}_t} \left(R_{i,t}^{(h)} \right).$$

We apply thresholded labeling with neutrality band $\tau = \texttt{label_tau} = 0.002$:

$$y_{i,t} = \begin{cases} 1, & R_{i,t}^{(h)} - m_t > \tau, \\ 0, & R_{i,t}^{(h)} - m_t < -\tau, \\ \text{NA}, & |R_{i,t}^{(h)} - m_t| \le \tau, \end{cases}$$

where neutral samples (NA) are excluded from both training and testing. This design reduces noise from near-median outcomes and focuses the classifier on clearer cross-sectional winners and losers.

Rolling train–test blocks. Let $\{t_1, \ldots, t_T\}$ be the ordered unique dates available after filtering and target construction. We apply a rolling block scheme with training length T_{train} and test length T_{test} (with an additional tail block if enough data remain):

$$\text{Block } b: \quad \mathcal{T}_{\text{train}}^{(b)} = \{t_s, \ldots, t_{s+T_{\text{train}}-1}\}, \qquad \mathcal{T}_{\text{test}}^{(b)} = \{t_{s+T_{\text{train}}}, \ldots, t_{s+T_{\text{train}}+T_{\text{test}}-1}\}.$$

In each block, the classifier is fit on $\mathcal{T}_{\text{train}}^{(b)}$ and evaluated on $\mathcal{T}_{\text{test}}^{(b)}$; train/test accuracies are recorded for diagnostic purposes.

Daily ranking and long–short portfolio construction. On each test day t, the fitted model outputs a score $p_{\text{up}}(i,t)$ for each $i \in \widetilde{\mathcal{U}}_t$. We sort by $p_{\text{up}}(i,t)$ and form a market-neutral portfolio with k longs and k shorts:

$$\mathcal{L}_t = \text{Top-}k \text{ contracts by } p_{\text{up}}(i,t), \qquad \mathcal{S}_t = \text{Bottom-}k \text{ contracts by } p_{\text{up}}(i,t).$$

A signal generated on date t is executed starting from date $t+1$ and held for h overnight return segments, producing overlapping sub-portfolios when $h > 1$.

Strategy return aggregation. Let $r_{i,u}^{\text{oo}}$ denote the realized overnight return segment on day u. The daily strategy return is computed as:

$$r_u^{\text{strat}} = \frac{1}{|\mathcal{L}_u|} \sum_{i \in \mathcal{L}_u} r_{i,u}^{\text{oo}} \; - \; \frac{1}{|\mathcal{S}_u|} \sum_{i \in \mathcal{S}_u} r_{i,u}^{\text{oo}},$$

where $\mathcal{L}_u$ and $\mathcal{S}_u$ include all positions that are active on day u due to the h-day holding rule.

Performance metrics and risk-free adjustment. We report annualized return, annualized volatility, maximum drawdown, win rate (computed on non-zero return days), and Sharpe ratio using an annual risk-free rate $r_f = 1.2\%$:

$$\text{Sharpe} = \frac{\text{AnnRet} - r_f}{\text{AnnVol}}, \qquad \text{AnnRet} = (1 + \overline{r})^{252} - 1, \qquad \text{AnnVol} = \sigma(r)\sqrt{252}.$$

To avoid distortion from the initial non-trading warm-up period, we optionally compute performance on the *real trading window* (from the first non-zero strategy return to the last), and separately diagnose the fraction of flat (zero-return) trading days.

Relative optimal hyper-parameters from experiments. Using the same protocol above, we conduct a small hyper-parameter search for each model and report the relatively optimal configuration used in the main experiments:

- **Random Forest (RF):**

$$\{\texttt{n_estimators} = 1000,\ \texttt{max_depth} = 12,\ \texttt{min_samples_leaf} = 100,\ \texttt{max_features} = \texttt{sqrt}\}.$$

- **XGBoost (XGB):**

$$\{\texttt{n_estimators} = 500,\ \texttt{max_depth} = 6,\ \texttt{learning_rate} = 0.05\}.$$

- **MLP (NN):** (with a compact configuration analogous to a small boosting ensemble)

$$\{\texttt{max_iter} = 50,\ \alpha = 10^{-4},\ \texttt{learning_rate_init} = 5 \times 10^{-3}\}.$$

4 Results

4.1 Single-Contract Backtest Performance (RF)

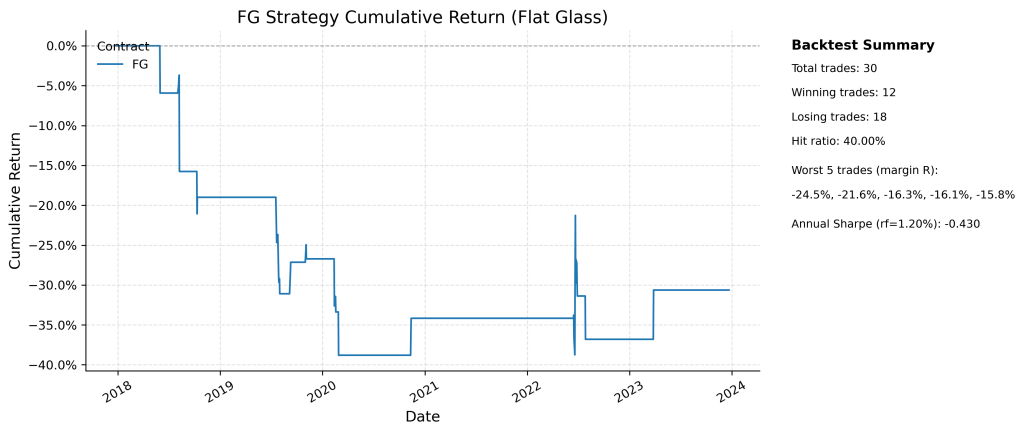

Figure 1: Flat Glass Contract

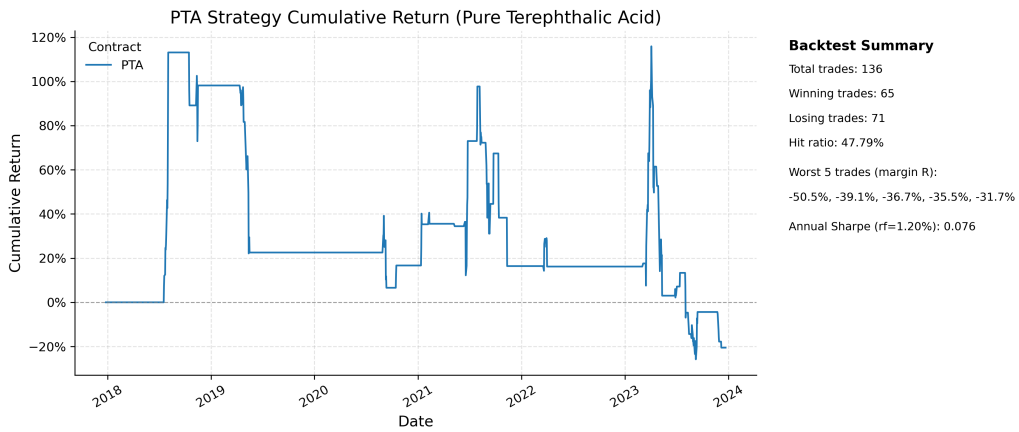

Figure 2: Pure Terephthalic Acid Contract

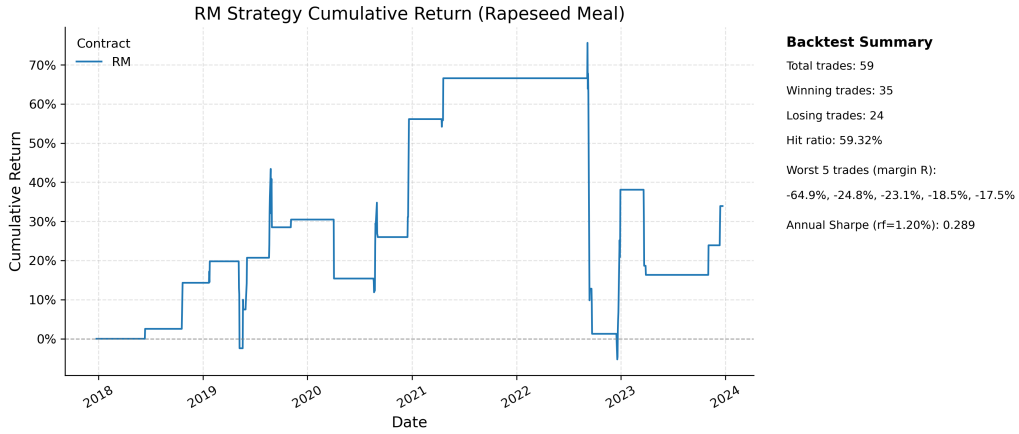

Figure 3: Rapeseed Meal Contract

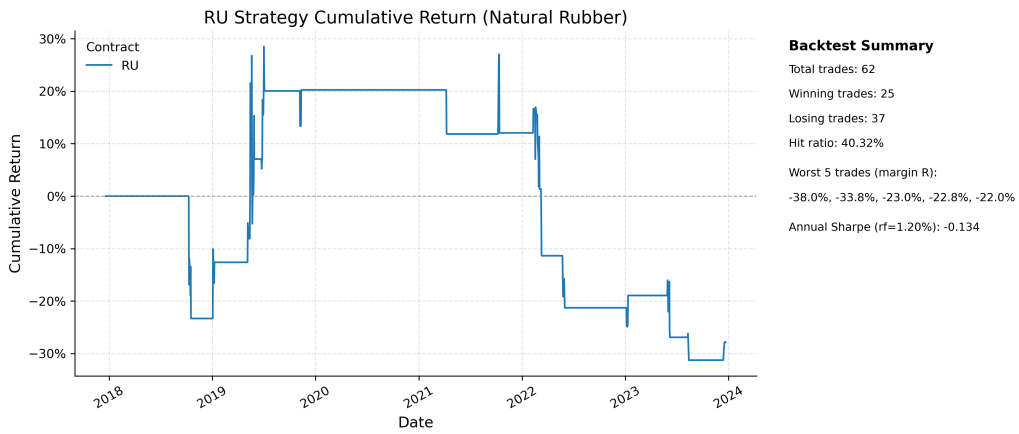

Figure 4: Natural Rubber Contract

4.2 Single-Contract Backtest Performance (XGB)

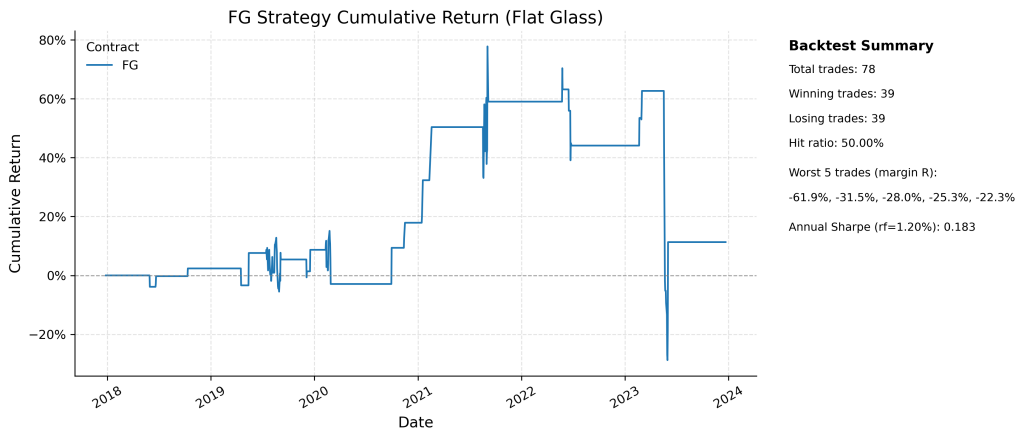

Figure 5: Flat Glass Contract

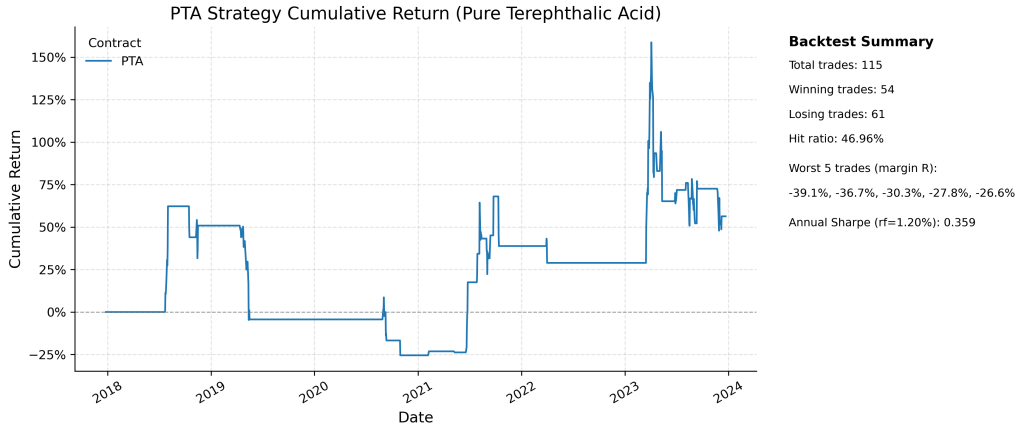

Figure 6: Pure Terephthalic Acid Contract

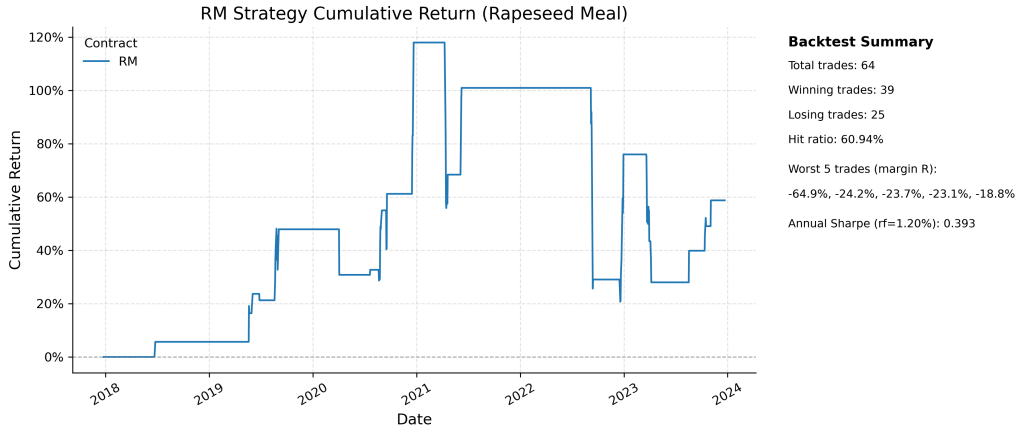

Figure 7: Rapeseed Meal Contract

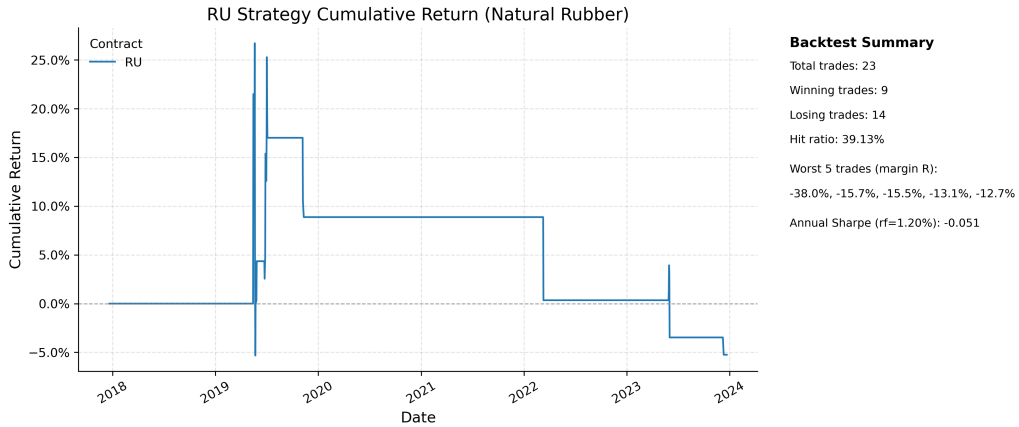

Figure 8: Natural Rubber Contract

4.3 Single-Contract Backtest Performance (MLP)

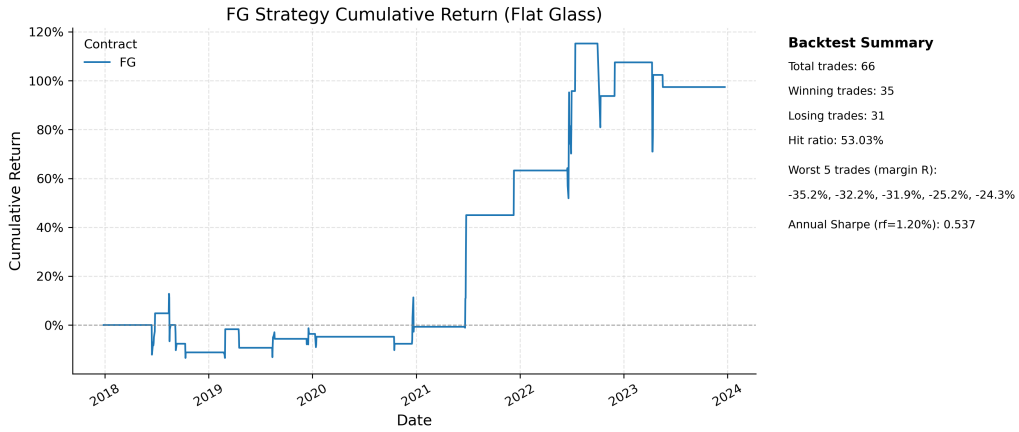

Figure 9: Flat Glass Contract

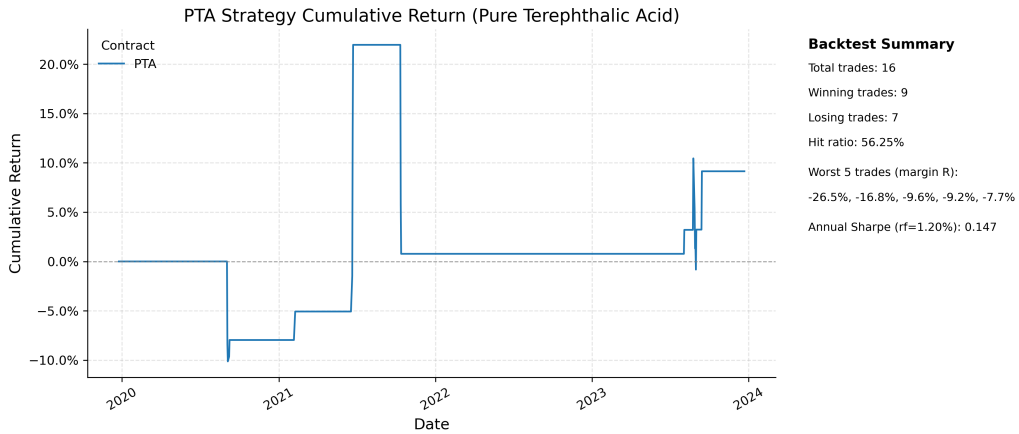

Figure 10: Pure Terephthalic Acid Contract

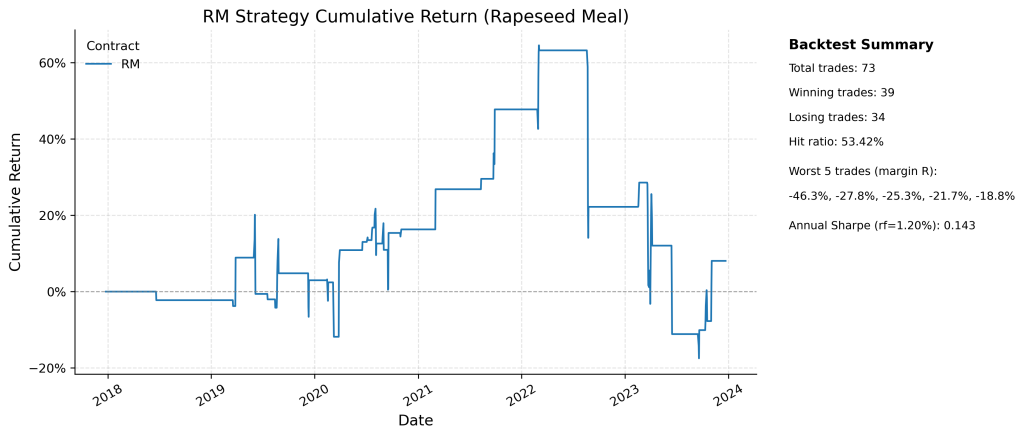

Figure 11: Rapeseed Meal Contract

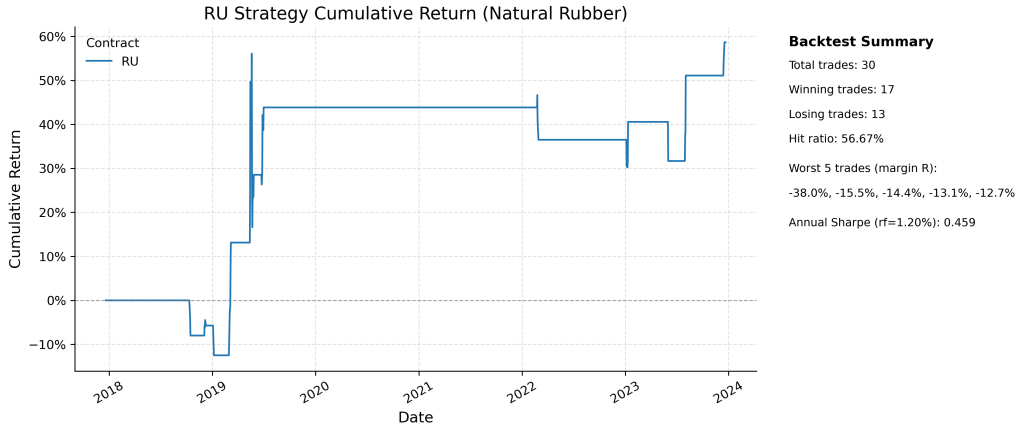

Figure 12: Natural Rubber Contract

4.4 Multiple-Contract Backtest Performance

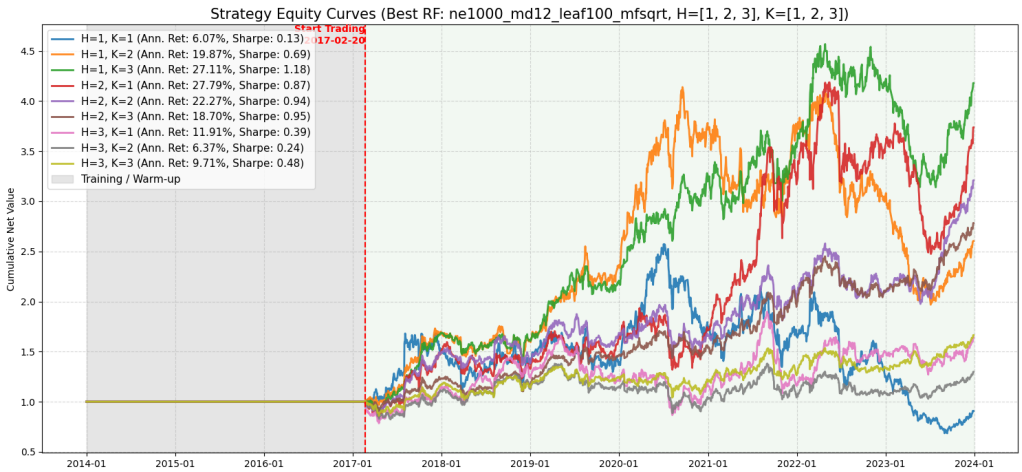

Figure 13: Random Forest (Equity Curves)

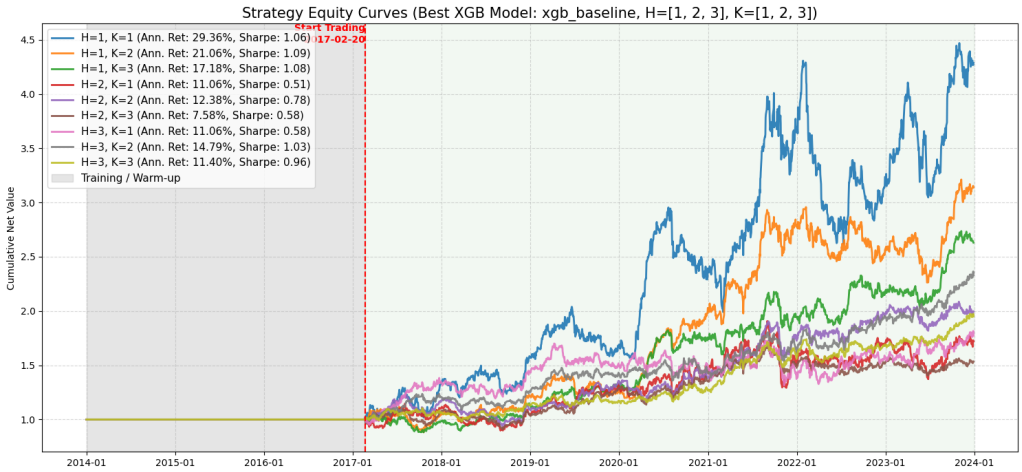

Figure 14: XGBoost (Equity Curves)

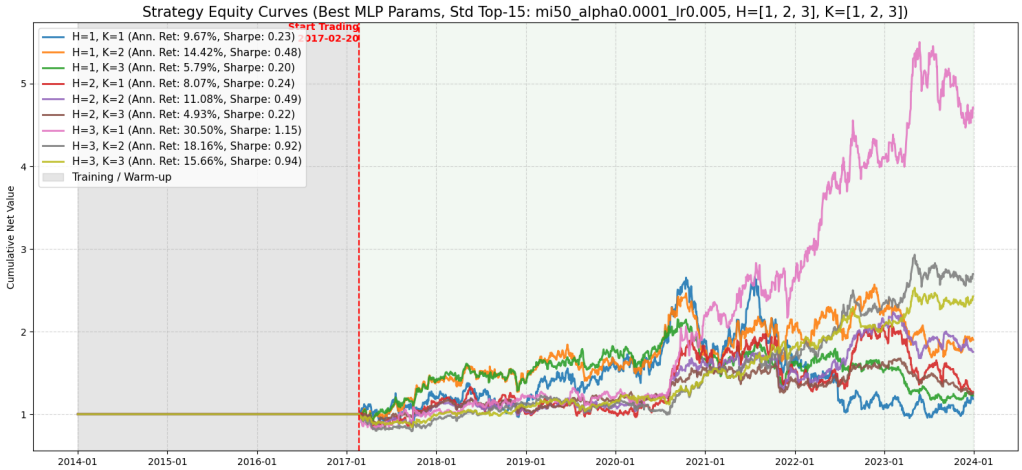

Figure 15: MLP (Equity Curves)

5 Summary

5.1 Models Comparison on Single Contract

Aspect	RF	XGB	MLP
Average return	FG: -5.1%/yr PTA: -3.6%/yr RM: 2.7%/yr RU: -5.2%/yr	FG: 1.9%/yr PTA: 7.6%/yr RM: 6.9%/yr RU: -0.6%/yr	FG: 16.2%/yr PTA: 2.3%/yr RM: 1.5%/yr RU: 9.8%/yr
Total trades	FG: 30 PTA: 136 RM: 59 RU: 62	FG: 78 PTA: 115 RM: 64 RU: 23	FG: 66 PTA: 16 RM: 73 RU: 30
Winning ratio	FG: 40.00% PTA: 47.79% RM: 59.32% RU: 40.32%	FG: 50.00% PTA: 46.96% RM: 60.94% RU: 39.13%	FG: 53.03% PTA: 56.25% RM: 53.42% RU: 56.67%
Worst 3 trades (margin R)	FG: -24.5,-21.6,-16.3% PTA: -50.5,-39.1,-36.7% RM: -64.9,-24.8,-23.1% RU: -38.0,-33.8,-23.0%	FG: -61.9,-31.5,-28.0% PTA: -39.1,-36.7,-30.3% RM: -64.9,-24.2,-23.7% RU: -38.0,-15.7,-15.5%	FG: -35.2,-32.2,-31.9% PTA: -26.5,-16.8,-9.6% RM: -46.3,-27.8,-25.3% RU: -38.0,-15.5,-14.4%
Annual Sharpe (rf=1.20%)	FG: -0.430 PTA: 0.076 RM: 0.289 RU: -0.134	FG: 0.183 PTA: 0.359 RM: 0.393 RU: -0.051	FG: 0.537 PTA: 0.147 RM: 0.143 RU: 0.459

Contract	Best	2nd	3rd
FG	**MLP**	XGB	RF
RU	**MLP**	XGB≈RF	–
RM	**XGB**	RF	MLP
PTA	**XGB**	MLP	RF

- **MLP** clearly rescues **FG/RU** (Sharpe ≈ 0.5, smoother P&L).
- **XGB** delivers the highest returns on **RM/PTA** but with the largest single-trade losses and drawdowns.
- **RF** only provides moderate alpha on **RM**; almost no edge on **FG/RU/PTA**.

Takeaway: No single model dominates. Multi-contract factor selection and strategy construction tend to deliver more stable and higher-return profiles than single-contract setups.

5.2 Models Comparison on Multiple Contract

Model	Ann. Ret	Sharpe	Max Drawdown	Win Rate
XGBoost	17.45%	0.9678	-21.60%	53.17%
Random Forest	27.12%	1.1801	-30.01%	52.92%
MLP (NN)	30.50%	1.1506	-23.42%	52.01%

Table 1: Performance comparison of three classification-based long/short models (best configuration).

Overall ranking. In risk-adjusted terms (Sharpe), Random Forest is the best (1.1801), closely followed by MLP (1.1506), while XGBoost ranks third (0.9678). In absolute return terms, MLP delivers the highest annualized return (30.50%), slightly above Random Forest (27.12%), and materially above XGBoost (17.45%). In downside risk terms (max drawdown), XGBoost is the most defensive (-21.60%), MLP is intermediate (-23.42%), and Random Forest experiences the deepest drawdown (-30.01%). Win rates are all near 52–53%, indicating that most of the performance differences come from the payoff distribution (average win/loss and tail behavior), not from hit-rate.

Why these differences are plausible.

- **Random Forest: best Sharpe but worst drawdown.** Bagging reduces variance and tends to produce stable cross-sectional rankings, which can improve Sharpe. However, RF probability estimates can become regime-dependent (especially when features are noisy but persistent), and in stressed periods the long/short ranking may "flip" together across many symbols, leading to larger drawdowns despite good average performance.
- **XGBoost: lowest drawdown but lower return/Sharpe.** Boosting with shrinkage and regularization often yields more conservative decision boundaries and smoother scores. This can limit exposure to extreme tails (hence smaller drawdown), but it may also under-exploit weaker nonlinear alpha signals, resulting in lower annualized return and a lower Sharpe than the stronger-performing configurations.
- **MLP: highest return with moderate drawdown.** MLPs can model richer nonlinear interactions and conditional effects (e.g., momentum $\times$ volatility states), which may increase capture of strong trends and improve absolute return. At the same time, neural nets are sensitive to scaling, class imbalance, and non-stationarity; even when Sharpe remains high, they can generate fatter-tailed P&L due to more aggressive score dispersion, producing drawdowns that are typically between RF and XGB.

Interpretation. Given the similar win rates, the key differentiator is **how each model allocates confidence** (score dispersion) and **how that confidence behaves across regimes**. RF and MLP appear to concentrate more in high-conviction bets (higher return, but RF shows worse tail risk here), while XGB appears more risk-controlled but potentially underfit relative to the best-performing RF/MLP settings.

5.3 Future Works

1. **Higher-frequency data and execution**
   - Current backtest is purely end-of-day with weekly overlays; no 15m/30m/5m/1m intraday data.

- This forces wide stops and allows large intraday drawdowns; a stop triggered on day t is only executable at the open of $t+1$.

2. **Richer factor space beyond pure price action**
   - Current factor set is dominated by technical / price-based signals.
   - Next iterations should incorporate fundamentals (inventory, production, imports/exports), term structure, and funding/leverage proxies.

3. **More structured score construction**
   - The current score is a flat aggregation of signals.
   - A more institutional design groups factors into sleeves (trend, momentum, reversal, value/growth, liquidity, sentiment) with explicit weights.

4. **More robust factor-selection criteria**
   - Selection is mainly driven by in-sample AUC/accuracy.
   - Add stability and tail-performance diagnostics: out-of-sample persistence, precision-at-k (e.g., top 1%/5%), and contribution to P&L and drawdowns.

5. **Cross-contract vs. single-contract factor combination**
   - Current experiments are split into single-contract and multi-contract settings.
   - A panel / hierarchical framework can recover common cross-contract factors while allowing contract-specific components.

6. **Refining exit rules: TP/SL/pullback/time stop**
   - Current TP/SL, pullback, and time-stop rules remain heuristic.
   - Parameterize and tune them via walk-forward or Bayesian search under turnover and tail-risk constraints.

References

1. Fabian Waldow, Matthias Schnaubelt, Christopher Krauss, and Thomas Günter Fischer (2021). *Machine Learning in Futures Markets.* Journal of Risk and Financial Management Article.

2. Masteika, Saulius, Rutkauskas V. Aleksandras, and Janes Andrea Aleksandra (2012). *Continuous futures data series for back testing and technical analysis.* Conference Proceedings, 3rd International Conference on Financial Theory and Engineering.

3. Nakajima, Tadahiro (2019). *Data structures for statistical computing in Python.* Paper presented at the 9th Python in Science Conference, Austin, TX, USA, June 28–July 3; vol. 445, pp. 56–61.

4. Vojtko, Radovan, and Matus Padysak (2020). *Continuous Futures Contracts Methodology for Backtesting.* Available online: https://ssrn.com/abstract=3517736.

5. Simon, David P. (1999). *The soybean crush spread: Empirical evidence and trading strategies.* Journal of Futures Markets.